

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ УПРАВЛІННЯ ФІНАНСАМИ ТА БІЗНЕСУ**

Кафедра цифрової економіки та бізнес-аналітики

КУРСОВА РОБОТА

з моделювання та автоматизації бізнес-процесів

на тему:

Інформаційна система для майстерні з ремонту та пошиття одягу

спеціальність: 051«Економіка»

(код та найменування спеціальності)

спеціалізація: «Інформаційні технології в бізнесі»

(найменування спеціалізації)

освітній ступінь бакалавр

(бакалавр/магістр)

Науковий керівник:

к.е.н., доцент Ярема О.Р.

(науковий ступінь, посада, прізвище, ініціали)

“ ”

20__ р.

(підпис)

Виконавець:

студент(ка) групи УФЕ-31С

Савчук А.О.

(прізвище, ініціали)

“ ”

20__ р.

(підпис)

Загальна кількість балів _____

(підпис, ППП членів комісії)

(підпис, ППП членів комісії)

(підпис, ППП членів комісії)

ЛЬВІВ 2026

ЗМІСТ

ВСТУП	3
РОЗДІЛ 1. АНАЛІТИЧНЕ ОБҐРУНТУВАННЯ ТА МОДЕЛЮВАННЯ ПРОЦЕСІВ МАЙСТЕРНІ	5
1.1. Аналіз ринку та маркетингове дослідження споживчих потреб	5
1.2. Опис та моделювання основного бізнес-процесу (BPMN)	8
1.3. Функціональні вимоги та UML-модель прецедентів	11
1.4. Моделювання потоків даних (DFD) та взаємодії з базою даних	13
РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РОЗРОБКА БАЗИ ДАНИХ АТЕЛЬЄ	18
2.1. Концептуальна модель даних	18
2.2. Застосування правил нормалізації	20
2.3. Характеристика типів даних	21
2.4. Впровадження обмежень цілісності	23
2.5. Програмування SQL-запитів для генерації звітів	24
РОЗДІЛ 3. ПРАКТИЧНА РЕАЛІЗАЦІЯ ВЕБ-ІНТЕРФЕЙСУ ТА БІЗНЕС-ЛОГІКИ	30
3.1. Архітектурна мапа та ключові функціональні блоки сайту	30
3.2. Програмна реалізація клієнтської частини (Frontend): стилізація та інтерактивність	37
3.3. Програмна реалізація серверної частини (Backend)	39
3.4. Реалізація модуля аналітичних звітів	42
3.5. Хостинг та розгортання проєкту	44
ВИСНОВКИ	45
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	47

ВСТУП

Стрімкий розвиток цифрових технологій диктує нові правила для підприємств сфери побутового обслуговування. Для сучасних ательє та кравецьких майстерень перехід від паперових журналів обліку до спеціалізованого програмного забезпечення є критично важливим кроком для виживання та масштабування бізнесу. Впровадження комплексної інформаційної системи дозволяє не лише пришвидшити обробку замовлень, але й налагодити точний облік специфічних даних, таких як анатомічні мірки клієнтів, залишки фурнітури та тканин, а також контроль завантаженості кожного окремого майстра.

Високий рівень конкуренції у сегменті індивідуального пошиття та ремонту одягу змушує власників шукати нові шляхи підвищення лояльності аудиторії. Автоматизація таких рутинних процесів, як бронювання часу на примірку, онлайн-калькуляція вартості робіт, розсилка сповіщень про готовність виробу та ведення фінансової звітності, здатна докорінно змінити якість сервісу. Система зводить до мінімуму ризик втрати інформації (наприклад, специфічних побажань клієнта щодо крою), виключає помилки при розрахунку витрат матеріалів та робить ціноутворення абсолютно прозорим.

Метою цієї курсової роботи є комплексне дослідження, проектування та практична розробка інформаційної системи, орієнтованої на потреби клієнтів і персоналу кравецької майстерні.

Об'єктом дослідження виступають процеси цифровізації послуг у сфері індивідуального пошиття.

Предметом дослідження є інструментарій та технології створення реляційних баз даних (SQL) та веб-додатків (HTML, Tailwind CSS, JavaScript, Python) для потреб ательє.

Для досягнення поставленої мети було визначено наступні **завдання**:

- Проаналізувати стан ринку швейних послуг та маркетингові тренди, а також провести емпіричне дослідження потреб цільової аудиторії для формування вимог до системи.
- Здійснити комплексне моделювання операційних процесів майстерні з використанням нотації BPMN, визначити функціональні ролі користувачів (UML) та спроектувати логічні потоки даних (DFD).
- Спроектувати та нормалізувати реляційну базу даних, впровадивши обмеження цілісності для забезпечення точності обліку анатомічних мірок, складських запасів та фінансових транзакцій.
- Розробити набір аналітичних SQL-алгоритмів для генерації управлінських звітів (ABC-аналіз послуг, KPI персоналу, моніторинг заборгованостей).
- Програмно реалізувати веб-інтерфейс системи, що включає публічний портал з інтерактивним калькулятором, кабінет клієнта з модулем трекінгу та CRM-панель адміністратора.
- Визначити параметри розгортання та хостингу інформаційної системи для забезпечення її доступності у хмарному середовищі.

Практична цінність розробки полягає у створенні готового до впровадження програмного продукту (ERP/CRM-платформи), здатного перевести роботу традиційного ательє у цифровий формат. Система забезпечує автоматизацію повного циклу обслуговування: від інтерактивної лідогенерації та цифровізації специфічних клієнтських даних (мірок) до формування глибокої бізнес-аналітики, що дозволяє суттєво оптимізувати витрати часу персоналу та підвищити конкурентоспроможність майстерні.

РОЗДІЛ 1. АНАЛІТИЧНЕ ОБҐРУНТУВАННЯ ТА МОДЕЛЮВАННЯ ПРОЦЕСІВ МАЙСТЕРНІ

1.1. Аналіз ринку та маркетингове дослідження споживчих потреб

Сфера послуг з ремонту та пошиття одягу наразі переживає період значної трансформації, зумовленої зміною економічних умов та глобальними екологічними трендами. Зниження купівельної спроможності змушує населення переходити до парадигми раціонального споживання, де реставрація гардероба виступає вигідною альтернативою купівлі нових речей. Окрім економічного фактора, потужним драйвером ринку став тренд на «апсайклінг» (upcycling)-переробку вживаного одягу, що, згідно з дослідженнями британської організації WRAP (Waste & Resources Action Programme), дозволяє зменшити екологічний слід від текстильної промисловості на 20-30 % за умови подовження терміну носіння речі хоча б на 9 місяців[1].

Для формування чіткого розуміння бізнес-вимог до майбутньої інформаційної системи було проведено анкетування 42 респондентів[2]. Результати дослідження дозволили сформувати детальний портрет цільової аудиторії: переважну більшість клієнтів складають жінки (77,5 %) віком від 18 до 35 років (понад 70 %). Це вказує на те, що основний споживач є «цифровим» і очікує можливості взаємодіяти з майстернею через смартфон.

Основними мотивами звернення до кравців виступають неможливість підібрати готовий одяг за особливостями фігури (37,5 %) та бажання замовити індивідуальне пошиття (35 %) (рис. 1.1). Серед найпопулярніших масових послуг лідирують заміна блискавок (76,2 %), припасування по фігурі (73,8 %) та дрібний ремонт (71,4 %). Саме на цих процесах має акцентуватися увага при проектуванні інтерфейсу замовлень.

Основна причина звернення до ательє
40 відповідей



Рис. 1.1 Основні причини звернення споживачів до майстерні

Джерело: побудовано на основі власних досліджень автора за результатами анкетного опитування українців у січні 2026 року

Щодо очікувань від сервісу, найважливішими критеріями вибору ательє клієнти назвали зручне розташування та графік (57,1 %), прозорість цін (54,8 %), а також наявність реальних відгуків (45,2 %) (рис. 1.2). Окрім цього, зафіксовано високий запит на цифрові послуги: 75% хочуть отримувати автоматичні сповіщення про статус замовлення, 52,5 % потребують функції онлайн-запису на примірку, а 45 % обирають онлайн-оплату. Ці дані є прямим обґрунтуванням необхідності впровадження багатофункціонального веб-додатка.

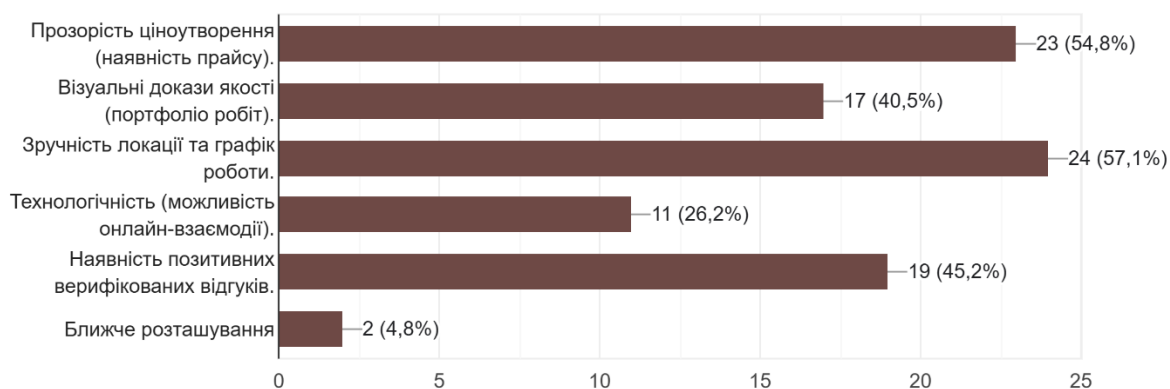


Рис. 1.2 Параметри сервісу для порівняльного аналізу конкурентоспроможності майстерні

Джерело: побудовано на основі власних досліджень автора за результатами анкетного опитування українців у січні 2026 року

Окрему увагу заслуговує ставлення до апсайклінгу: понад 85 % респондентів позитивно сприймають ідею переробки старого одягу (Рис. 1.3). Це відкриває можливість формування іміджу майстерні як сучасного та екологічно відповідального бізнесу.



Рис.1.3 Оцінка інтересу респондентів до послуги апсайклінгу

Джерело: побудовано на основі власних досліджень автора за результатами анкетного опитування українців у січні 2026 року

Підсумкове технічне завдання (ТЗ) на розробку інформаційної системи

На основі проведеного аналізу ринку та виявлених ключових потреб споживачів, було сформовано концептуальне технічне завдання. Інформаційна система майстерні має проєктуватися як комплексна ERP/CRM-платформа з розподілом ролей та складатися з трьох базових модулів:

1. Публічний веб-портал (Front-end для лідогенерації):

- **Інтерактивний калькулятор послуг:** для задоволення потреби клієнтів у «прозорості цін» (54,8 % опитаних). Калькулятор повинен динамічно враховувати тип виробу, складність тканини та терміновість, з можливістю миттєвого переходу до оформлення заявки.
- **Блок «Sustainable Fashion» (Апсайклінг):** створення окремого розділу для популяризації екологічного напрямку (запит 85 % респондентів).
- **Інформаційні блоки:** інтеграція модулів з відгуками клієнтів, графіком роботи та контактами.

2. Особистий кабінет клієнта (Client Portal):

- **Трекінг замовлень:** реалізація візуального відстеження статусу пошиття («Прийнято», «В роботі», «На примірці», «Готово») для задоволення потреби у контролі процесу (75 % опитаних).
- **Платіжний шлюз:** інтеграція модуля безпечної онлайн-оплати послуг (вибір банківської картки або електронних гаманців).

3. Панель адміністратора (CRM-система):

- **Дашборд керівника:** виведення ключових показників ефективності (KPI) в режимі реального часу (загальний дохід, середній чек, кількість активних замовлень).
- **Управління бізнес-процесами:** можливість оперативно створювати нові замовлення, змінювати їх статуси та призначати відповідальних майстрів.
- **Автоматизований контроль складу:** система системних сповіщень про критичний залишок витратних матеріалів (фурнітури, тканин).

Саме ці визначені вимоги лягли в основу проєктування архітектури бази даних та розробки програмного коду, що детально описано в наступних розділах роботи.

1.2. Опис та моделювання основного бізнес-процесу (BPMN)

Для забезпечення безперебійної роботи майстерні необхідно чітко алгоритмізувати етапи взаємодії з клієнтом, починаючи від першого звернення і закінчуючи видачею готового виробу. Сучасний підхід до управління ательє передбачає строгий контроль за рухом матеріалів, фіксацію мірок та моніторинг стадій виробництва.

Для візуалізації операційної діяльності була використана міжнародна нотація BPMN (Business Process Model and Notation) згідно з офіційною специфікацією [15]. Розроблена діаграма чітко розмежовує зони відповідальності між трьома ключовими учасниками процесу: Клієнтом, Менеджером та Майстром (Рис. 1.4).

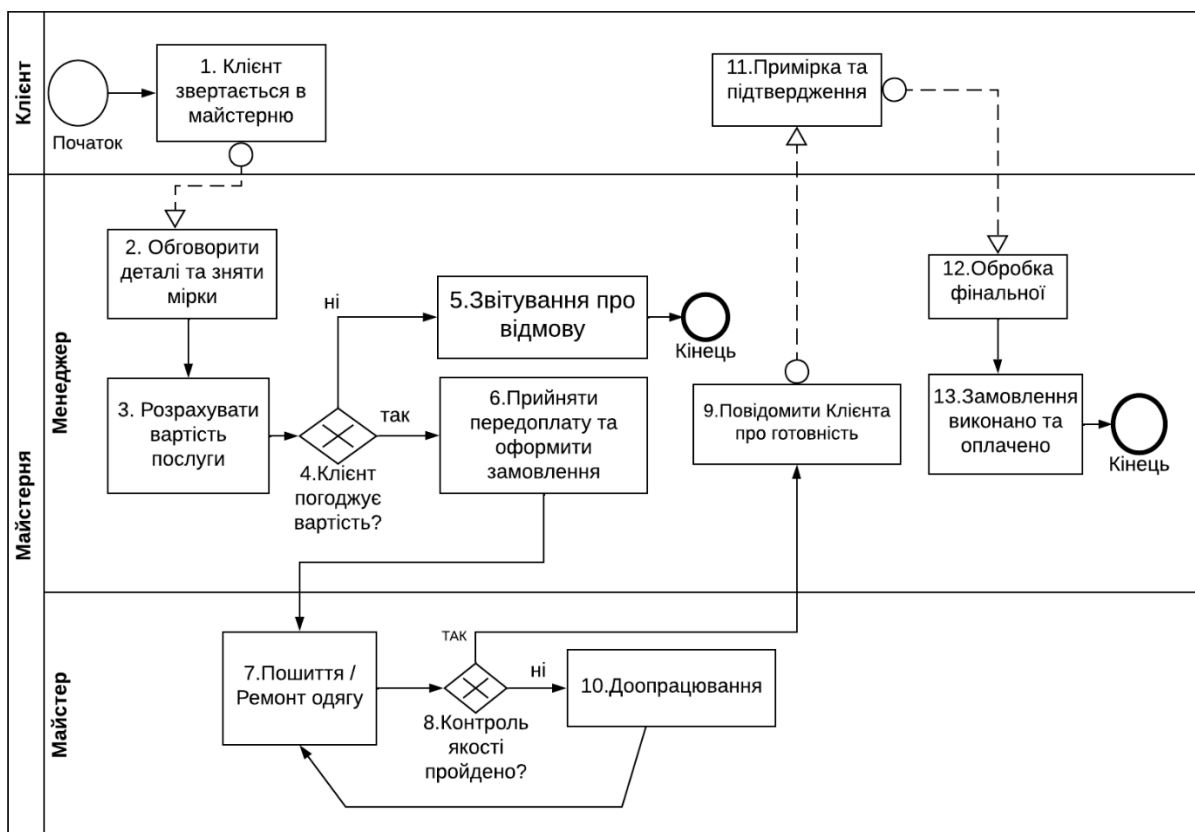


Рис. 1.4 BPMN-діаграма бізнес-процесу

Джерело: авторська розробка

Відповідно до моделі, основний бізнес-процес “Прийом та виконання замовлення” розгортається за наступним сценарієм:

Процес стартує з моменту звернення клієнта (1). Першою дією Менеджера є "Обговорити деталі та зняти мірки" (2).

- *Аналіз:* цей етап є фундаментом якості. Помилка тут (невірні заміри, нечітко записані побажання) не може бути виправлена на наступних етапах без значних втрат. Це вимагає від менеджера не лише комунікативних навичок, але й базового розуміння технології пошиття.

Далі слідує "Розрахувати вартість послуги" (3). Це внутрішня аналітична операція, що базується на прайс-листі та оцінці складності робіт, зробленої на попередньому кроці.

Етап 2: комерційні переговори та фінансова фіксація (Gateway 4, Tasks 5–6)

Наступає перший критичний момент прийняття рішення - шлюз "Клієнт погоджує вартість?" (4). Тут потік розділяється:

- Сценарій "Ні": відбувається перехід до завдання "Звітування про відмову" (5). Процес завершується.
- *Значення:* наявність цього шляху в моделі дозволяє аналізувати конверсію звернень у замовлення. Велика кількість переходів по цій гілці може сигналізувати про неконкурентні ціни.
- Сценарій "Так": виконується дія "Прийняти передоплату та оформити замовлення" (6).
- *Значення:* введення передоплати є інструментом управління фінансовими ризиками. Це гарантує покриття витрат на матеріали та підтверджує серйозність намірів клієнта. Оформлення замовлення (документування) запускає виробничий цикл.

Етап 3: виробництво та управління якістю (Tasks 7–8, 10)

Після оформлення, управління передається на доріжку "Майстер". Виконується центральне завдання - "Пошиття / Ремонт одягу" (7). Одразу після завершення роботи настає етап валідації - шлюз "Контроль якості пройдено?" (8).

- Сценарій "Ні" (Цикл переробки): якщо виявлено дефекти, процес переходить до завдання "Доопрацювання" (10), після чого стрілка повертається назад до шлюза контролю.
- *Глибинний аналіз:* ця петля (Loop) є механізмом захисту репутації. Вона гарантує, що неякісний продукт не покине межі виробничої зони. Однак, з економічної точки зору, кожна ітерація цього циклу збільшує собівартість послуги (подвійні витрати часу майстра) без збільшення ціни для клієнта. Мінімізація потрапляння в цю петлю є головним завданням технічного менеджменту.

Етап 4: фіналізація та закриття угоди (Tasks 9, 11–13)

При успішному проходженні контролю ("Так"), ініціатива повертається до Менеджера.

1. "Повідомити Клієнта про готовність" (9): активується потік повідомлень до клієнта.
2. "Примірка та підтвердження замовлення" (11): клієнт прибуває в майстерню для фінальної перевірки. Це момент істини для клієнта.
3. "Обробка фінальної оплати" (12): клієнт сплачує залишок суми.
4. "Замовлення виконано та оплачено" (13): фінальна подія, що закриває екземпляр процесу.

1.3. Функціональні вимоги та UML-модель прецедентів

Інформаційна система ательє базується на принципі суворого розмежування прав доступу, що дозволяє убезпечити комерційні дані та структурувати роботу персоналу. Для проектування архітектури взаємодії користувачів із системою була побудована UML-діаграма прецедентів (Use Case)[4], у якій виділено чотирьох основних акторів: Відвідувач, Клієнт, Адміністратор та Кравець(рис. 1.5).

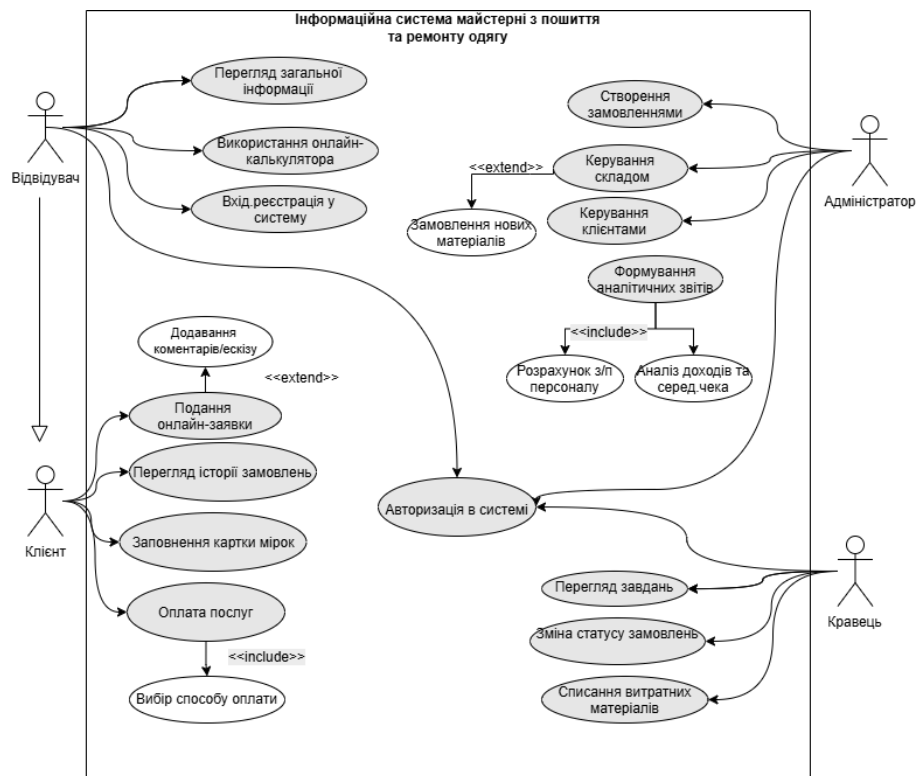


Рис.1.5 UML-модель прецедентів

Джерело: авторська розробка

Детальний розподіл ролей та прецедентів згідно з розробленою моделлю виглядає так:

Відвідувач (неавторизований юзер): має обмежений, ознайомчий доступ. Його базові функції включають «Перегляд загальної інформації» (прайс-лист, контакти, портфоліо) та «Використання онлайн-калькулятора» для самостійного орієнтовного прорахунку послуг. Також він ініціює прецедент «Вхід/реєстрація у систему», щоб набути статусу клієнта.

Клієнт (авторизований юзер): на діаграмі пов'язаний із Відвідувачем відношенням узагальнення (наслідування), що означає успадкування всіх базових функцій. Отримавши доступ до закритого особистого кабінету, його головною дією стає «Подання онлайн-заявки», яка має опціональне розширення (відношення <<extend>>) - «Додавання коментарів/ескізу», що вкрай важливо для індивідуального пошиття. Окрім цього, клієнт здійснює «Перегляд історії замовлень» та самостійне «Заповнення картки мірок». Важливим елементом є прецедент «Оплата послуг», який безальтернативно включає в себе (відношення <<include>>) дію «Вибір способу оплати».

Адміністратор: як і інші ролі персоналу, обов'язково взаємодіє з прецедентом «Авторизація в системі». Після входу він отримує найширші привілеї для управління бізнесом. Саме адміністратор відповідає за «Створення замовлення» (фізичне формування запису в базі даних на основі заявок клієнтів). Основні завдання також включають «Керування замовленнями» та «Керування клієнтами» (редагування бази, аналіз лояльності). Прецедент «Керування складом» може динамічно доповнюватися дією «Замовлення нових матеріалів» (залежність <<extend>>), якщо запаси тканини вичерпано. Крім того, адміністратор відповідає за «Формування аналітичних звітів», що обов'язково базується на двох підпроцесах (через <<include>>): «Розрахунок з/п персоналу» та «Аналіз доходів та середнього чека».

Кравець: виступає як виробнича одиниця системи. Після «Авторизації в системі» майстер виконує «Перегляд завдань», що призначені саме йому. В процесі роботи він здійснює «Зміну статусу замовлень» (наприклад, переводить

у статус "Готово") та проводить «Списання витратних матеріалів» зі складу майстерні.

Таке глибоке моделювання функціоналу гарантує, що майбутня система повністю задовільнить потреби як споживачів, так і управлінської ланки ательє.

1.4. Моделювання потоків даних (DFD) та взаємодії з базою даних

Для логічного зв'язку між діями користувачів (акторів) та внутрішньою структурою зберігання інформації було розроблено діаграму потоків даних (Data Flow Diagram)[5].

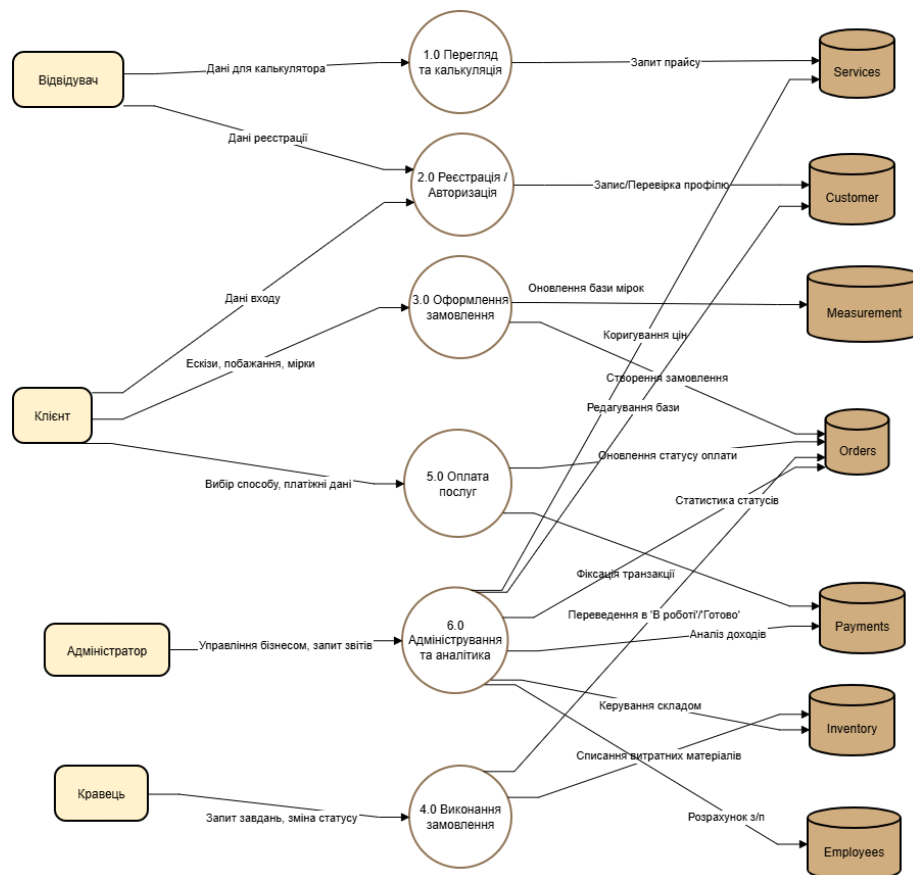


Рис.1.6 Data Flow Diagram

Джерело: авторська розробка

Ця модель візуалізує процеси переміщення, обробки та збереження даних на кожному етапі роботи майстерні, об'єднуючи інтерфейс з архітектурою бази даних. Архітектура потоків даних базується на шести ключових процесах, які безпосередньо взаємодіють із сімома основними сховищами інформації (базами

даних) та реалізуються через комплекс спеціалізованих SQL-запитів (детальніше описаних у підрозділі 2.5):

1.0 Перегляд та калькуляція: відвідувач взаємодіє із системою, вводячи дані для калькулятора (тип виробу, складність, матеріали). Процес здійснює динамічний «Запит прайсу» (через базовий SELECT запит) з таблиці Services для виведення актуальної та орієнтовної вартості на екран.

2.0 Реєстрація/Авторизація: відвідувач (передаючи дані реєстрації) або існуючий Клієнт (передаючи дані входу) проходять процедуру ідентифікації. Процес забезпечує «Запис або перевірку профілю» у центральній таблиці Customer (використовуючи DML-операції INSERT та SELECT).

3.0 Оформлення замовлення: клієнт формує технічне завдання, передаючи ескізи, побажання та власні мірки. Система діє багатовекторно: здійснює «Оновлення бази мірок» у таблиці Measurement (для миттєвого підняття найсвіжіших антропометричних даних клієнта тут використовується складний вкладений підзапит) та ініціює безпосереднє «Створення замовлення» у таблиці Orders.

4.0 Виконання замовлення: цей процес обслуговує Кравець, який надсилає запит на отримання завдань та фіксує зміну їх стану. Процес автоматично здійснює «Переведення замовлення в статус 'В роботі' або 'Готово'» у таблиці Orders, а також виконує «Списання витратних матеріалів» (ниток, фурнітури) з таблиці Inventory (контроль критичних залишків на складі реалізовано через функцію COALESCE та багатотабличні об'єднання).

5.0 Оплата послуг: клієнт обирає спосіб оплати та вводить платіжні дані. Після успішного списання коштів, процес здійснює «Оновлення статусу оплати» у таблиці Orders та «Фіксацію транзакції» у таблиці фінансового обліку Payments. Для моніторингу цього етапу розроблено аналітичний механізм контролю заборгованостей клієнтів (з використанням LEFT JOIN та умови HAVING).

6.0 Адміністрування та аналітика: адміністратор, здійснюючи управління бізнесом та запит звітів, використовує цей комплексний макропроцес, який має найширший доступ до системи. Він дозволяє:

- здійснювати «Коригування цін» у базі Services ;
- проводити «Редагування бази» клієнтів Customer та виявляти VIP-клієнтів (за допомогою загальних табличних виразів CTE);
- збирати «Статистику статусів» з Orders для аналізу конкретного виду робіт ,наприклад екологічного напрямку (Upcycling);
- виконувати порівняльний «Аналіз доходів» та рентабельності на основі Payments (із застосуванням умовної логіки CASE WHEN та агрегатних функцій SUM, DATE_FORMAT);
- формувати статистику продуктивності для «Розрахунку з/п» на базі таблиці Employees (KPI персоналу).

Побудована DFD-модель є фундаментальним містком між бізнес-вимогами та технічною реалізацією. Вона жорстко регламентує, які саме дані генеруються на кожному етапі, де вони зберігаються та якими саме SQL-алгоритмами обробляються. Це слугує прямим обґрунтуванням для проектування нормалізованої реляційної бази даних та інтеграції аналітичних звітів, детально описаних у Розділі 2.

1.5. Обґрунтування вибору технологічного стека

Створення цифрової платформи для кравецької майстерні вимагає ретельного підбору інструментів, здатних забезпечити високу швидкість розробки, надійність зберігання даних та приємний користувацький досвід. Система логічно поділяється на середовище розробки, клієнтську та серверну частини.

Середовище розробки (IDE):

В якості основного інструменту написання програмного коду було обрано Visual Studio Code (VS Code). Цей вибір є максимально обґрунтованим, оскільки VS Code позиціонується як швидкий, легкий та кросплатформний редактор, який

ідеально підходить для розробки веб-застосунків. Серед його ключових переваг варто виділити вбудовану систему розумного автодоповнення коду (IntelliSense), що суттєво пришвидшує написання HTML-розмітки та скриптів. Наявність інтегрованого термінала та засобів роботи з системою контролю версій Git безпосередньо у вікні редактора оптимізує робочий процес, дозволяючи розробнику не відволікатися на сторонні програми. Завдяки потужній екосистемі розширень, VS Code дозволяє зручно налаштувати середовище під будь-які мови програмування (як для фронтенду, так і для бекенду).

Клієнтська частина (Front-end):

Для реалізації візуальної частини застосовується сучасний підхід до верстки інтерфейсів:

- HTML5[7] формує базовий семантичний каркас сторінок (відповідно до розробленої структури, це секції про апсайклінг, послуги, особистий кабінет).
- Tailwind CSS (Utility-first CSS)[8]: замість написання громіздких класичних таблиць стилів, обґрунтованим є використання фреймворку Tailwind CSS. Це передове рішення дозволяє стилізувати елементи безпосередньо в HTML-тегах за допомогою готових утилітарних класів (наприклад, `flex`, `flex-col`, `bg-brand-dark`, `text-brand-gold`, `font-serif`). Такий підхід робить код більш компактним, пришвидшує розробку кастомного та адаптивного дизайну, а також гарантує автоматичну оптимізацію розміру фінального файлу стилів для швидкого завантаження сайту.
- JavaScript[9] відповідає за клієнтську інтерактивність: логіку роботи онлайн-калькулятора вартості, плавну фільтрацію портфолію та миттєву валідацію форм без перезавантаження сторінки.

Серверна частина (Back-end) та База даних:

Серверна архітектура будується з використанням мови Python (на базі мікрофреймворку Flask з точкою входу через файл `app.py`)[10]. Python є надзвичайно потужним інструментом для швидкої бекенд-розробки, який

дозволяє легко розгортати REST API, обробляти бізнес-логіку авторизації та керувати алгоритмами складського обліку тканин.

Фундаментом для збереження даних обрано реляційну базу даних під управлінням MySQL. Оскільки процеси ательє генерують значний масив пов'язаних між собою сутностей (клієнт пов'язаний із специфічними мірками, замовленням та списанням конкретної фурнітури), саме реляційна модель MySQL гарантує абсолютну транзакційну цілісність, захищає від дублювання та дозволяє ефективно генерувати складну фінансову аналітику.

талії (waist), стегон (hips), довжина рукава (sleeve_length) та ширина спини (cross_back).

Центральним вузлом виробничого процесу є сутність **Orders** (Замовлення). Вона фіксує дату створення (order_date), очікувану дату видачі (expected_delivery_date), поточний стан виконання (status) та підсумкову вартість (total_amount). Кожне замовлення пов'язане з конкретним клієнтом та працівником (employee_id), який несе за нього відповідальність.

Для управління персоналом створено сутність **Employees** (Працівники), яка містить дані про імена, посади (position), графік роботи (schedule), фінансову ставку (rate) та статус активності співробітника (is_active).

Фінансові транзакції реєструються в таблиці **Payments** (Платежі), де вказані суми (amount), методи оплати (payment_method) та точний час транзакції (transaction_moment). Платіж прив'язується виключно до конкретного замовлення (order_id), що дозволяє уникнути надлишковості даних.

Оскільки одне замовлення може включати декілька видів послуг та вимагати використання різних матеріалів, архітектура бази даних містить розв'язувальні (асоціативні) таблиці для зв'язків "багато-до-багатьох":

1. **Order_Items:** зв'язує замовлення із довідником **Services** (Послуги, де зберігається service_name та base_price). Ця таблиця містить кількість замовлених послуг (quantity) та текстові побажання клієнта (comments).
2. **Order_Materials:** зв'язує замовлення із таблицею **Inventory** (Склад). Склад контролює наявність матеріалів (material_name, артикул sku, одиниці виміру unit, залишок stock_quantity, ціну за одиницю price_per_unit). Проміжна таблиця фіксує, яку саме кількість певного матеріалу було фактично витрачено на конкретне замовлення (used_quantity).

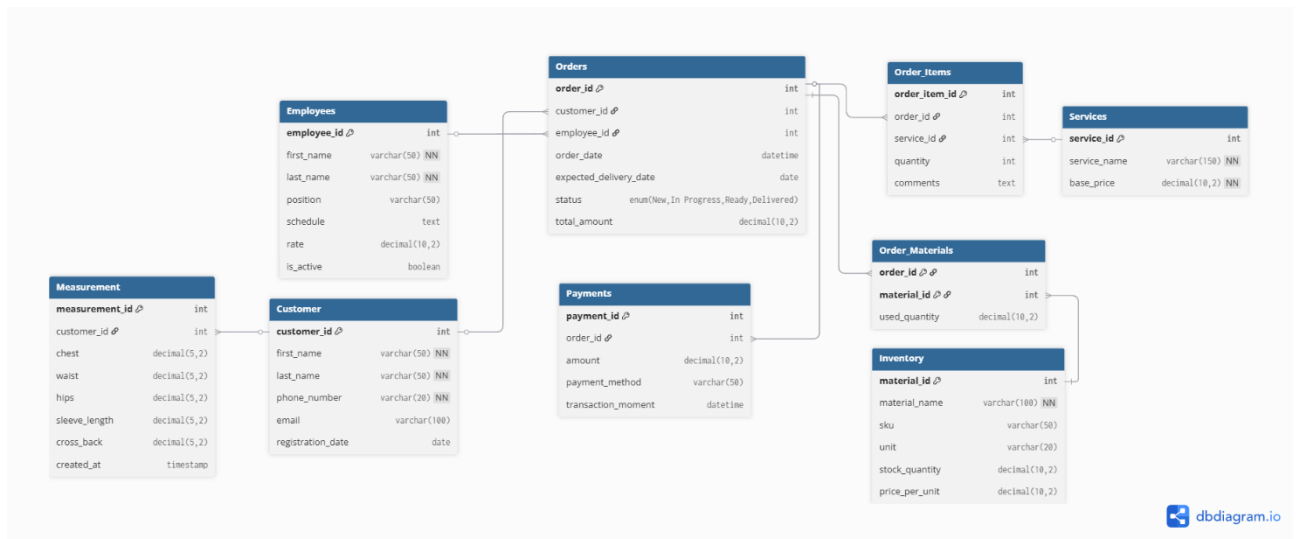


Рис.2.2 Реляційна схема бази даних

2.2. Застосування правил нормалізації

Для забезпечення швидкодії СУБД MySQL та уникнення дублювання інформації, структура бази даних пройшла процес строгої нормалізації[11].

Перша нормальна форма (1НФ): усі атрибути містять виключно атомарні значення. Наприклад, ім'я та прізвище клієнта розділені на окремі поля `first_name` та `last_name`. Замість того, щоб вписувати всі анатомічні параметри в одне текстове поле, кожен показник (талія, груди) винесено в окремий числовий стовпець таблиці `Measurement`.

Друга нормальна форма (2НФ): жоден неключовий атрибут не залежить від частини складеного ключа. Наприклад, у розв'язувальній таблиці `Order_Materials` кількість витраченого матеріалу (`used_quantity`) повністю залежить від комбінації первинних ключів `order_id` та `material_id`. Назва самого матеріалу при цьому не дублюється, а береться з таблиці `Inventory`.

Третя нормальна форма (3НФ): повністю усунуто транзитивні залежності. Таблиця `Orders` не містить контактних даних замовника або імені майстра - вона зберігає лише їхні унікальні ідентифікатори (`customer_id`, `employee_id`). Це гарантує, що при зміні номера телефону клієнта, інформацію потрібно буде оновити лише в одному місці, запобігаючи виникненню аномалій оновлення.

2.3. Характеристика типів даних

Раціональний вибір типів даних у спроектованій базі даних є важливим чинником забезпечення ефективності її функціонування, зокрема оптимізації використання дискового простору, підвищення швидкості виконання запитів та забезпечення цілісності даних. Відповідно до структури предметної області майстерні, у базі даних використано стандартні типи даних, регламентовані офіційною документацією рушія MySQL [12] (табл. 2.3-2.5).

Текстові типи даних (табл. 2.3) застосовуються для збереження символічної інформації змінної довжини. Зокрема, для зберігання імен, прізвищ та посад працівників використовується тип VARCHAR із обмеженням довжини до 50 символів, що є достатнім для даного виду інформації та дозволяє уникнути перевитрат пам'яті. Для назв послуг, матеріалів та електронних адрес використано VARCHAR більшої довжини (100–150 символів), що забезпечує гнучкість при введенні даних різного обсягу. Для збереження великих текстових блоків, таких як коментарі до замовлень або опис графіку роботи, застосовується тип TEXT, який дозволяє зберігати значні обсяги тексту. Фіксація статусів виконання замовлення реалізована за допомогою типу ENUM, що дозволяє обмежити набір можливих значень і, таким чином, підвищити узгодженість та коректність даних.

Таблиця 2.3

Текстові типи даних

Назва	Опис
VARCHAR(50)	Використовується для збереження імен, прізвищ та посад працівників.
VARCHAR(100)	Застосовується для назв послуг і матеріалів.
VARCHAR(150)	Використовується для електронних адрес клієнтів.
TEXT	Призначений для збереження довгих текстів (коментарі, графік роботи).

Назва	Опис
ENUM	Використовується для статусів: 'New', 'In Progress', 'Fitting', 'Ready', 'Delivered', 'Cancelled'.

Числові типи даних (табл. 2.4) використовуються для представлення кількісних та ідентифікаційних показників. Усі первинні та зовнішні ключі реалізовані на основі типу INT, що забезпечує ефективну ідентифікацію записів і підтримку зв'язків між таблицями. Для збереження фінансових показників, таких як ціни, суми платежів та вартість послуг, використовується тип DECIMAL(10,2), який забезпечує високу точність обчислень і виключає похибки округлення, характерні для типів із плаваючою комою. Для збереження анатомічних мірок клієнтів застосовується DECIMAL(5,2), що дозволяє фіксувати значення з необхідною точністю. Логічні характеристики, зокрема статус активності працівника, реалізовано за допомогою типу BOOLEAN.

Таблиця 2.4

Числові типи даних

Назва	Опис
INT	Використовується для первинних та зовнішніх ключів (ідентифікаторів).
DECIMAL(10,2)	Застосовується для фінансових показників (ціни, суми, платежі).
DECIMAL(5,2)	Використовується для анатомічних мірок тіла з точною деталізацією.
BOOLEAN	Логічний тип для збереження статусу активності працівника.

Типи дати і часу (табл. 2.5) використовуються для фіксації часових параметрів функціонування системи. Для збереження календарних дат, таких як дата реєстрації замовлення або встановлений дедлайн, застосовується тип DATE. Для подій, що потребують точної фіксації моменту часу (наприклад, створення замовлення або проведення фінансової операції), використовується тип DATETIME. Тип TIMESTAMP застосовується для автоматичної фіксації часу створення запису, що є важливим для ведення обліку та аудиту змін у системі

Таблиця 2.5

Типи дати і часу

Назва	Опис
DATE	Використовується для збереження дат (дедлайни, реєстрація).
DATETIME	Фіксує точну дату і час подій (замовлення, платежі).
TIMESTAMP	Використовується для автоматичної фіксації часу створення запису.

Детальніше розглянути типи даних та їх обмеження можна на ER-діаграмі з програми Workbench (рис. 2.3).

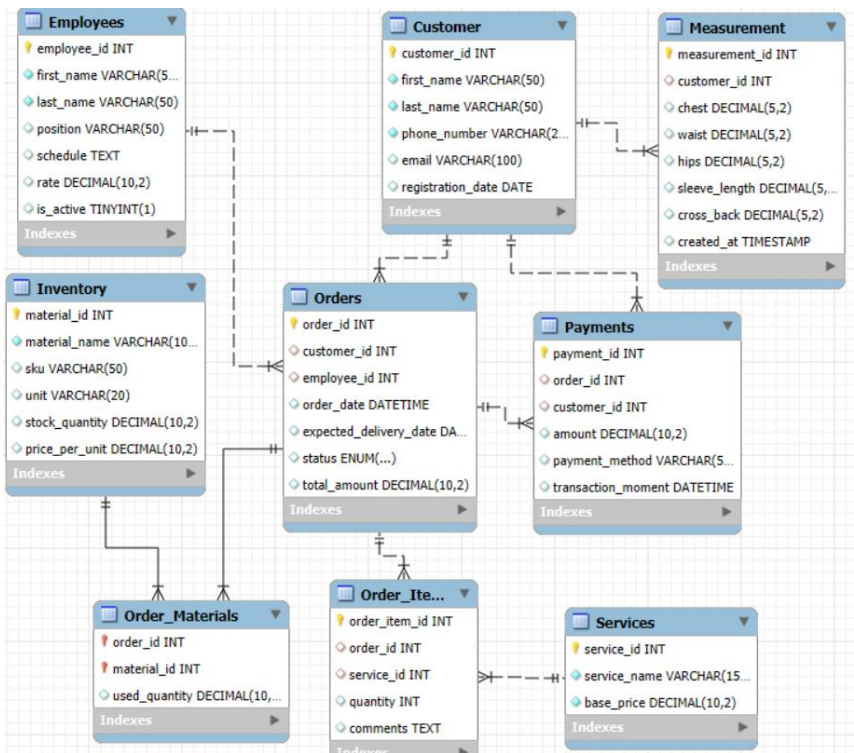


Рис. 2.3 ER-діаграма Workbench

2.4. Впровадження обмежень цілісності

Обмеження цілісності даних (Data Integrity Constraints) є ключовим інструментом СУБД для забезпечення точності та надійності збереженої інформації. Вони реалізовані через наступні механізми:

1. **Первинні ключі (PRIMARY KEY):** кожна таблиця має унікальний ідентифікатор, що генерується системою (наприклад, `order_id`, `material_id`), унеможливлюючи сплутування записів.
2. **Зовнішні ключі (FOREIGN KEY):** забезпечують реляційну цілісність. Якщо у таблиці `Orders` вказано `customer_id`, система жорстко перевіряє наявність такого клієнта. Розв'язувальні таблиці (`Order_Items`, `Order_Materials`) містять зовнішні ключі, що посилаються на відповідні батьківські таблиці, унеможливлюючи появу "осиротілих" записів.
3. **Обов'язкові поля (NOT NULL):** Згідно з фізичною моделлю (позначки NN), ключові поля, такі як `first_name`, `last_name`, `phone_number` у клієнтів, `service_name`, `base_price` у послугах та `material_name` у складі, категорично не можуть бути порожніми.
4. **Умови перевірки (CHECK):** використовуються для контролю бізнес-логіки. Наприклад, введено правило CHECK (`stock_quantity >= 0`) для складу тканин, що технічно блокує можливість утворення від'ємного залишку.

2.5. Програмування SQL-запитів для генерації звітів

Для оперативного управління майстернею розроблено набір агрегаційних SQL-запитів. Ці запити інтегровані у серверну логіку (через бібліотеку `mysql-connector-python`) і виконуються при відповідних діях адміністратора або клієнта.

1. Запит для підняття антропометричних даних клієнта

Демонструє використання вкладеного підзапиту (Subquery) для пошуку найсвіжіших записів. Дозволяє майстру миттєво отримати останні актуальні мірки клієнта перед початком розкрою, виключаючи ризик використання застарілих параметрів фігури (Рис. 2.4).


```

166 • SELECT c.first_name,
167         c.last_name,
168         m.chest,
169         m.waist,
170         m.hips,
171         m.created_at
172 FROM Customer c
173 JOIN Measurement m ON c.customer_id = m.customer_id
174 WHERE m.created_at = (
175     SELECT MAX(created_at)
176     FROM Measurement
177     WHERE customer_id = c.customer_id
178 );

```

first_name	last_name	chest	waist	hips	created_at
Анна	Савчук	92.00	68.00	96.00	2026-03-21 18:13:46
Марія	Ковальчук	88.00	64.00	92.00	2026-03-21 18:13:46
Олена	Мельник	100.00	80.00	105.00	2026-03-21 18:13:46
Світлана	Бондар	94.00	72.00	98.00	2026-03-21 18:13:46

Рис. 2.4 Антропометричні дані клієнта

2. Аналіз загальної прибутковості ательє по місяцях

Використовує агрегатні функції та форматування дат (DATE_FORMAT). Цей критично важливий для управління звіт дозволяє власнику відстежувати фінансові потоки в розрізі місяців та планувати бюджет майстерні (Рис. 2.5).

```

180 • SELECT DATE_FORMAT(transaction_moment, '%Y-%m') AS finance_month,
181           SUM(amount) AS total_revenue
182 FROM Payments
183 GROUP BY DATE_FORMAT(transaction_moment, '%Y-%m')
184 ORDER BY finance_month ASC;

```

finance_month	total_revenue
2026-03	5000.00

Рис. 2.5 Прибутковість ательє по місяцях

3. Рейтинг популярності та рентабельності послуг (ABC-аналіз)

Запит базується на багатотабличному об'єднанні (JOIN) з використанням математичних операцій. Допомогає виявити "послуги-локомотиви", які генерують найбільший дохід, що є базисом для прийняття маркетингових рішень (Рис. 2.6).

```

185 • SELECT s.service_name, SUM(oi.quantity) AS total_items_sold,
186       SUM(s.base_price * oi.quantity) AS generated_revenue
187 FROM Order_Items oi
188 JOIN Services s ON oi.service_id = s.service_id
189 GROUP BY s.service_name
190 ORDER BY generated_revenue DESC
191 LIMIT 5;

```

service_name	total_items_sold	generated_revenue
Пошиття вечірньої/корсетної сукні	1	4500.00
Пошиття костюма (жакет + брюки)	1	3000.00
Апсайклінг: перешиття піджака	1	1800.00
Індивідуальне пошиття сукні	1	1500.00

Рис. 2.6 Рейтинг послуг

5. Статистика продуктивності персоналу (KPI)

Багатовимірна вибірка з фільтрацією за діапазоном дат та множинними статусами. Слугує автоматизованою базою для розрахунку відрядної заробітної плати та преміювання працівників за фактично закриті замовлення (Рис. 2.7).

```

225 • SELECT CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
226       e.position,
227       COUNT(o.order_id) AS completed_tasks
228 FROM Employees e
229 JOIN Orders o ON e.employee_id = o.employee_id
230 WHERE o.status IN ('Ready', 'Delivered')
231       AND o.expected_delivery_date BETWEEN '2026-03-01' AND '2026-03-31'
232 GROUP BY e.employee_id, employee_name, e.position
233 ORDER BY completed_tasks DESC;

```

employee_name	position	completed_tasks
Ірина Ткач	кравець	3
Наталія Кравець	кравець	1
Ольга Стильна	закрійник	1

Рис. 2.7 Статистика продуктивності персоналу

6. Контроль заборгованостей клієнтів

Складний запит з використанням LEFT JOIN та умови HAVING. Порівнює загальну вартість замовлення з фактично внесеними платежами, виводячи список клієнтів, які мають непогашений борг (Рис. 2.8).

```

241 • SELECT o.order_id,
242         CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
243         o.total_amount,
244         COALESCE(SUM(p.amount), 0) AS paid_amount,
245         (o.total_amount - COALESCE(SUM(p.amount), 0)) AS debt
246 FROM Orders o
247 JOIN Customer c ON o.customer_id = c.customer_id
248 LEFT JOIN Payments p ON o.order_id = p.order_id
249 GROUP BY o.order_id, customer_name, o.total_amount
250 HAVING debt > 0
251 ORDER BY debt DESC;

```

order_id	customer_name	total_amount	paid_amount	debt
10	Марія Ковальчук	3000.00	0.00	3000.00
1	Анна Савчук	4500.00	2000.00	2500.00
8	Олена Мельник	1800.00	0.00	1800.00
4	Світлана Бондар	1800.00	0.00	1800.00
2	Марія Ковальчук	1500.00	0.00	1500.00
6	Марія Ковальчук	1200.00	0.00	1200.00
9	Анна Савчук	500.00	0.00	500.00
5	Ігор Петренко	500.00	0.00	500.00
7	Світлана Бондар	200.00	0.00	200.00

Рис. 2.8 Контроль заборгованостей клієнтів

7. Управління складом:

Аналітичний звіт, який вираховує, скільки матеріалу залишилося на складі після виконання замовлень. Незамінний інструмент для вчасного закупівлі нових тканин(Рис. 2.9).

```

253 • SELECT i.sku,
254         i.material_name,
255         i.stock_quantity AS initial_stock,
256         COALESCE(SUM(om.used_quantity), 0) AS totally_used,
257         (i.stock_quantity - COALESCE(SUM(om.used_quantity), 0)) AS remaining_stock
258 FROM Inventory i
259 LEFT JOIN Order_Materials om ON i.material_id = om.material_id
260 GROUP BY i.material_id, i.sku, i.material_name, i.stock_quantity
261 ORDER BY remaining_stock ASC;

```

sku	material_name	initial_stock	totally_used	remaining_stock
FAB-WOOL-002	Вовна жовта (Sunny)	8.00	4.00	4.00
FAB-SILK-001	Шовк італійський (чорний)	15.50	3.50	12.00
FAB-DEN-003	Джинс вінтажний	20.00	0.00	20.00
ACC-ZIP-01	Блискавка потайна 20см	30.00	0.00	30.00
THR-BLK-01	Нитки армовані (чорні)	50.00	1.00	49.00

Рис. 2.9 Управління складом

8. Виявлення VIP-клієнтів

Запит використовує загальні табличні вирази (СТЕ - WITH) для розрахунку середнього чека по всьому ательє, а потім знаходить тих клієнтів, чії витрати перевищують цей середній показник (Рис. 2.10).

```

264 • WITH AverageCheck AS (
265     SELECT AVG(total_amount) AS avg_amount FROM Orders
266 )
267 SELECT c.first_name, c.last_name,
268        COUNT(o.order_id) as total_orders,
269        SUM(o.total_amount) as total_spent
270 FROM Customer c
271 JOIN Orders o ON c.customer_id = o.customer_id
272 GROUP BY c.customer_id, c.first_name, c.last_name
273 HAVING SUM(o.total_amount) > (SELECT avg_amount FROM AverageCheck)
274 ORDER BY total_spent DESC;

```

first_name	last_name	total_orders	total_spent
Марія	Ковальчук	3	5700.00
Анна	Савчук	2	5000.00
Олена	Мельник	2	4800.00
Світлана	Бондар	2	2000.00

Рис.2.10 Виявлення VIP-клієнтів

9. Аналіз екологічного напрямку (Upscycling)

Спеціалізована вибірка, що шукає всі замовлення, пов'язані з переробкою або відновленням одягу. Використовує пошук за ключовими словами як у назві послуги, так і в коментарях до замовлення (Рис. 2.11).

```

276 • SELECT o.order_date, c.first_name, s.service_name, oi.comments
277 FROM Orders o
278 JOIN Order_Items oi ON o.order_id = oi.order_id
279 JOIN Services s ON oi.service_id = s.service_id
280 JOIN Customer c ON o.customer_id = c.customer_id
281 WHERE s.service_name LIKE '%Апсайклінг%'
282        OR oi.comments LIKE '%перешити%'
283        OR oi.comments LIKE '%переробити%';

```

order_date	first_name	service_name	comments
2026-03-18 11:00:00	Світлана	Апсайклінг: перешиття піджака	Переробити чоловічий піджак на жіночий ов...
2026-03-15 15:00:00	Олена	Апсайклінг: перешиття піджака	Перешиття піджака
2026-03-05 11:30:00	Марія	Апсайклінг: джинсовий корсет	Апсайклінг: джинсовий корсет зі старих джи...

Рис.2.11 Аналіз послуги Upscycling

10. Порівняльний аналіз напрямків бізнесу (Пошиття vs Еко-ремонт)

Аналітичний запит із використанням умовної логіки (CASE WHEN). Дозволяє власнику бізнесу оцінити рентабельність двох основних векторів роботи ательє: створення нового одягу та екологічного відновлення/підгонки існуючих речей (Рис. 2.12).

```

285 • SELECT
286     CASE
287         WHEN s.service_name LIKE '%Апсайклінг%'
288             OR s.service_name LIKE '%Заміна%'
289             OR s.service_name LIKE '%Підгонка%'
290         THEN 'Еко-напрямок та ремонт'
291         ELSE 'Індивідуальне пошиття'
292     END AS business_direction,
293     SUM(oi.quantity) AS items_processed,
294     SUM(s.base_price * oi.quantity) AS projected_revenue
295 FROM Order_Items oi
296 JOIN Services s ON oi.service_id = s.service_id
297 GROUP BY business_direction
298 ORDER BY projected_revenue DESC;

```

business_direction	items_processed	projected_revenue
Індивідуальне пошиття	4	12000.00
Еко-напрямок та ремонт	6	6000.00

Рис.2.12 Порівняльний аналіз напрямків бізнесу

11. Коригування цінової політики

DML-запит (Data Manipulation Language) для оперативного оновлення даних. Дозволяє провести швидку індексацію вартості певних категорій послуг (наприклад, через інфляцію) з використанням паттерн-матчингу (LIKE) (Рис. 2.13).

```

235 • UPDATE Services
236 SET base_price = base_price * 1.10
237 WHERE service_name LIKE '%ремонт%' OR service_name LIKE '%Заміна блискавки%';

```

#	Time	Action	Message	Duration / Fetch
24	10:08:54	SELECT CONCAT(e.first_name, ' ', e.last_name) ...	3 row(s) returned	0.000 sec / 0.000 sec

Рис.2.13 Коригування цінової політики

РОЗДІЛ 3. ПРАКТИЧНА РЕАЛІЗАЦІЯ ВЕБ-ІНТЕРФЕЙСУ ТА БІЗНЕС-ЛОГІКИ

3.1. Архітектурна мапа та ключові функціональні блоки сайту

Інформаційна система логічно поділена на декілька взаємопов'язаних модулів відповідно до розробленої ієрархічної мапи сайту (Sitemap)(Рис.3.1). Правильна організація гарантує інтуїтивно зрозумілу навігацію та безперебійний користувацький досвід. Для схематичного зображення карти сайту було використано програму draw.io.

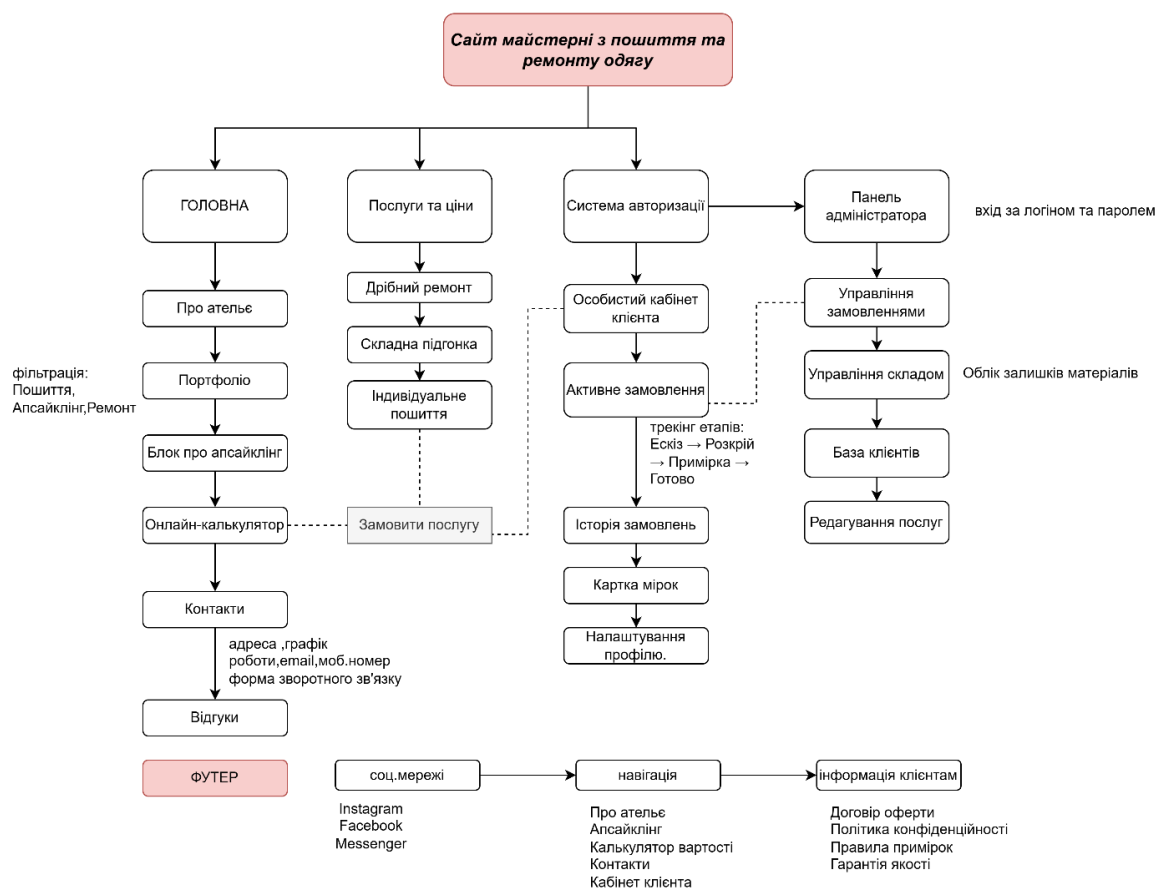


Рис. 3.1. Мапа сайту майстерні з ремонту та пошиття одягу

Джерело: Розроблено автором

Опис ключових функціональних блоків та макетів сторінок:

На основі розробленої мапи сайту було створено високоточні візуальні макети (Mockups) веб-додатку майстерні бренду «ELENIKO».

Головна сторінка (Landing Page): виконує презентаційну та маркетингову функції. Її відкриває головний банер (Hero Section) із гаслом "Створюємо одяг, що підкреслює вашу унікальність" та інтерактивними кнопками заклику до дії (Рис.3.2).

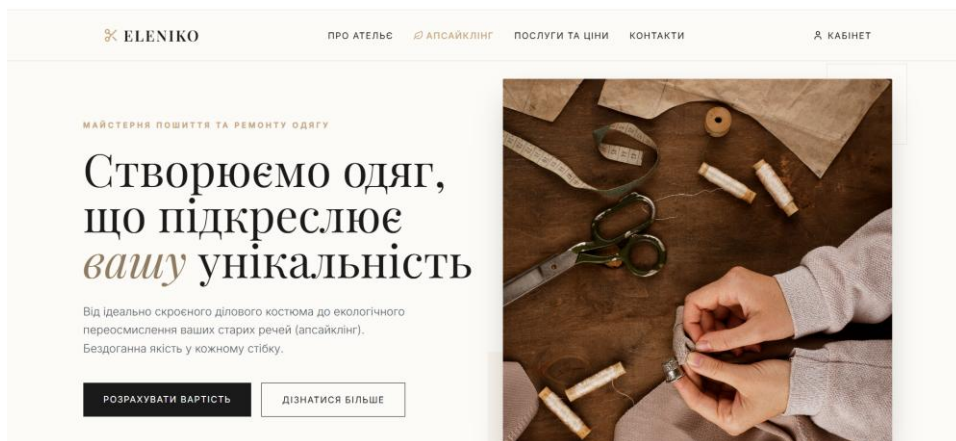


Рис.3.2 Landing page

Нижче розташований блок "Про ательє", який підкреслює експертність команди (10+ років досвіду, 1500+ задоволених клієнтів) та філософію сталого розвитку (Рис.3.3).

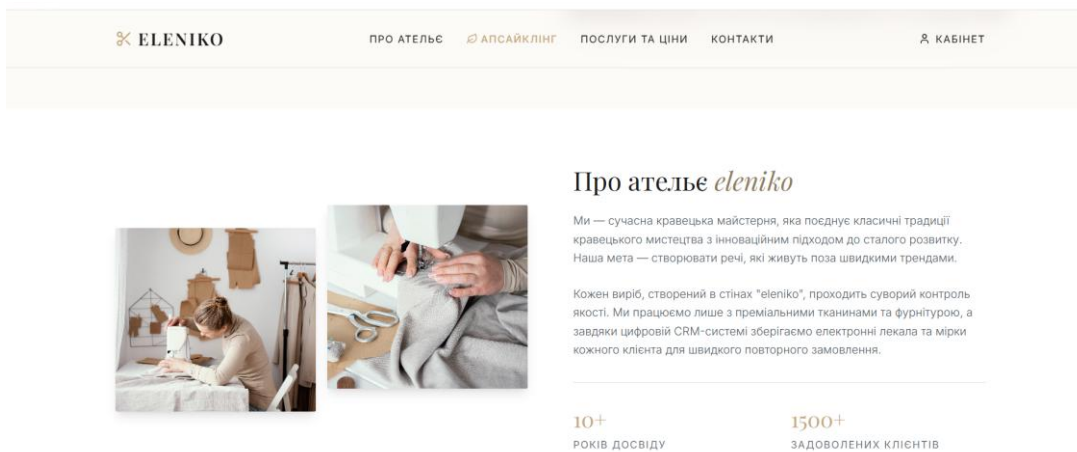


Рис.3.3 Landing page

Динамічне "Портфоліо" оснащено ефектами наведення (hover effects) та зручною системою вкладок для фільтрації категорій: "Усі роботи", "Пошиття", "Апсайклінг" (Рис.3.4).

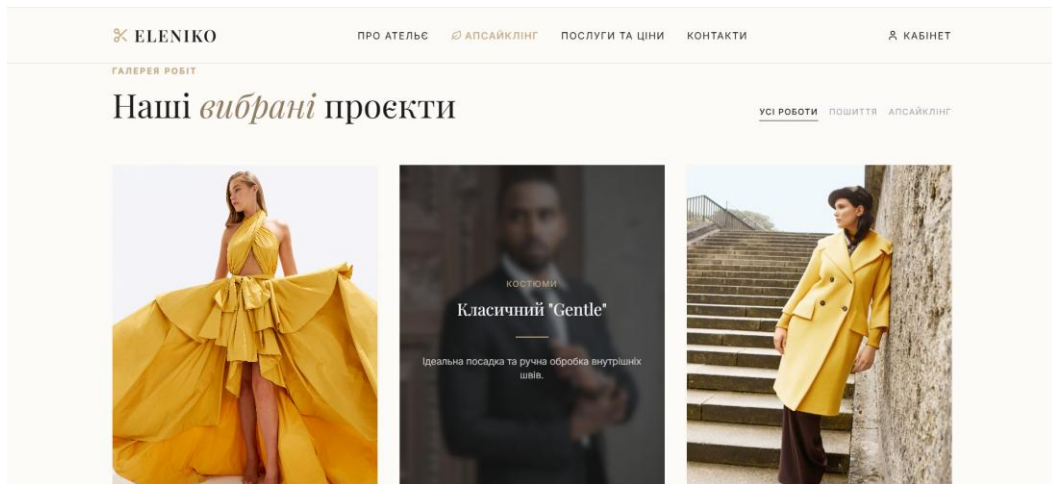


Рис.3.4 Портфоліо

Окремим візуальним акцентом є темна секція, присвячена апсайклінгу - "Нове життя улюблених речей", яка пояснює можливості трансформації одягу (Рис.3.5).

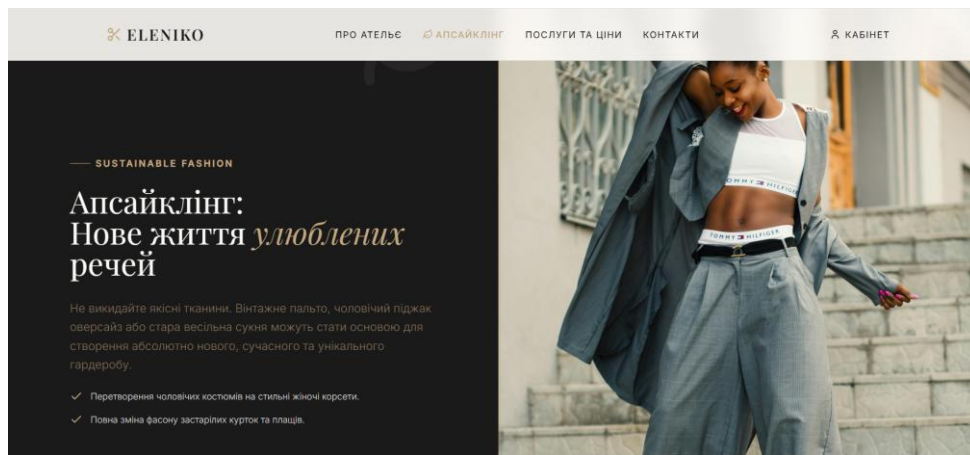


Рис.3.5 Блок про «Апсайклінг»

Найважливішим конверсійним інструментом головної сторінки є інтерактивний блок «Онлайн-калькулятор» (Рис. 3.6). Візуально його інтерфейс поділений на логічні кроки, що дозволяють користувачу самостійно сконструювати замовлення: обрати тип послуги (пошиття, апсайклінг, ремонт), вказати конкретний виріб, складність тканини та додаткові опції (наприклад, термінове виконання). Калькулятор миттєво виводить орієнтовну вартість на екран, що закриває потребу клієнтів у прозорості ціноутворення, виявлену під час аналізу ринку.

Рис.3.6 Онлайн-калькулятор вартості послуги

Для покращення воронки продажів (лідогенерації), інтерфейс калькулятора логічно поєднано з формою зворотного зв'язку: після розрахунку клієнт може в один клік перейти до оформлення заявки, куди автоматично підтягнуться обрані ним параметри.

Завершують сторінку секції "Відгуки наших клієнтів" у вигляді карток та блок "Наші контакти" із зазначенням локації, графіком роботи та формою для швидкого зворотного зв'язку (Рис.3.7).

Рис.3.7 Контакти, форма зворотного зв'язку та відгуки

Модуль "Послуги та ціни": відображає актуальний прайс-лист. Структура сторінки включає зручну бокову навігацію за категоріями ("Дрібний ремонт", "Складна підгонка", "Індивідуальне пошиття"). У центральній частині виведено перелік послуг із зазначенням орієнтовної вартості (Рис.3.8). У майбутньому цей блок буде динамічно генеруватися на основі даних з таблиці Services, що дозволить адміністратору змінювати ціни без втручання в програмний код сторінки.

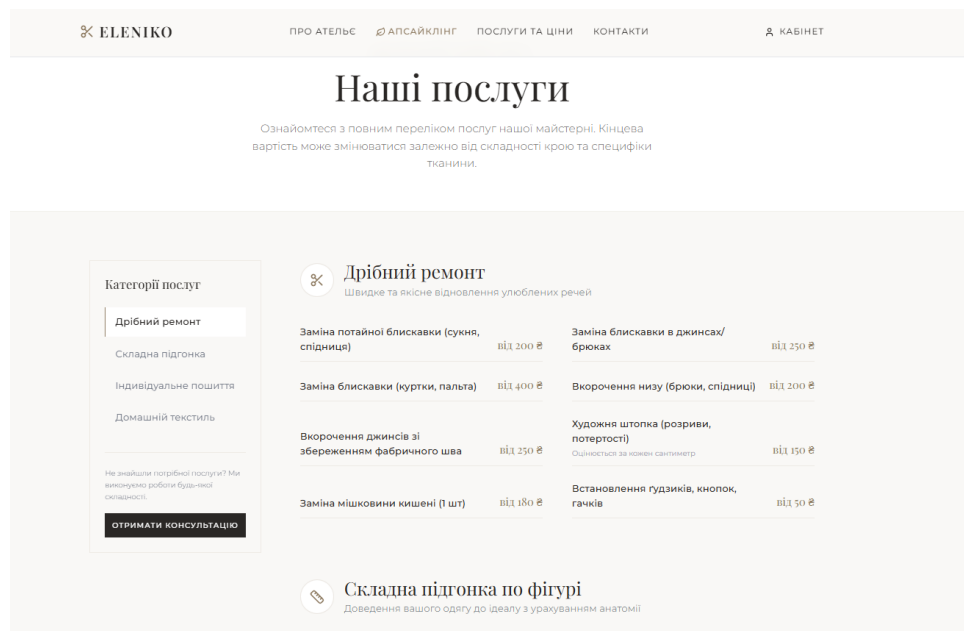


Рис.3.8 Послуги та ціни

Особистий кабінет клієнта: закрита частина сайту, доступна після авторизації. Цей блок генерується сервером динамічно на основі унікального ідентифікатора користувача (customer_id). Інтерфейс кабінету містить дашборд, який виводить:

- Картку "**Активне замовлення**" (наприклад, "Вечірня сукня з шовку", 4500 ₴) з інноваційним візуальним трекером етапів виробництва.
- Опцію онлайн-оплати, де клієнт може обрати спосіб транзакції (Банківська картка або Apple/Google Pay)
- Таблицю "**Історія замовлень**", що демонструє попередні звернення та їхній статус.

- Персональну "Картку мірок"- спеціально виділений віджет, який зберігає антропометричні дані клієнта (Обхват грудей, талії, стегон тощо) з можливістю їх оперативного оновлення онлайн (Рис.3.9).

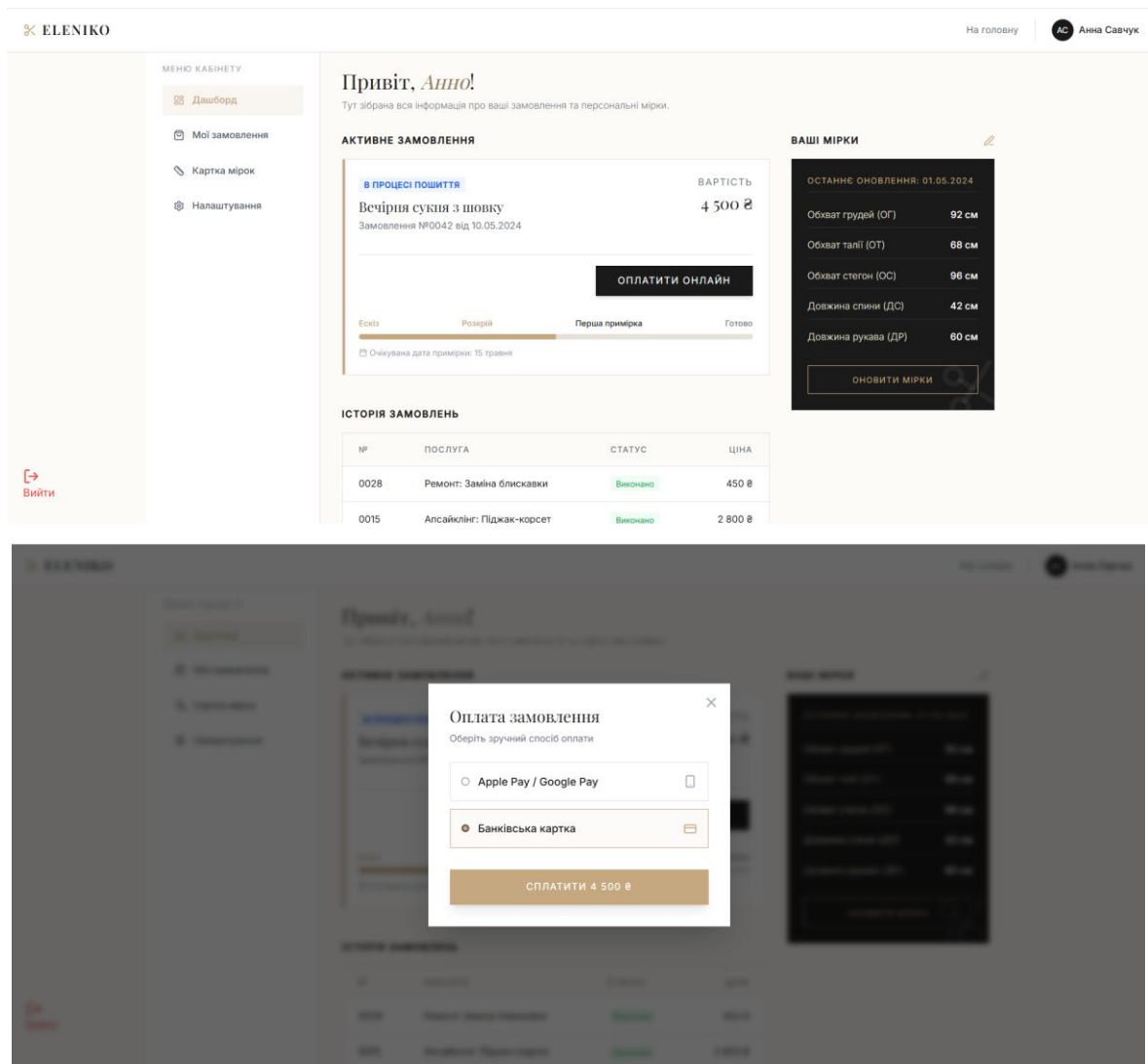


Рис.3.9 Особистий кабінет клієнта

Панель адміністратора (CRM): потужний модуль для оперативного управління бізнес-процесами, який надає адміністратору повний контроль над інформаційною системою. Головний «Дашборд» відображає ключові показники ефективності (KPI) у вигляді карток: загальний дохід за місяць, середній чек, кількість замовлень у роботі та автоматичні системні сповіщення (наприклад, «Матеріали закінчуються: 2 позиції»). Важливою архітектурною особливістю є те, що всі ці показники не є статичними, а розраховуються сервером динамічно в режимі реального часу на основі агрегаційних SQL-запитів до бази даних.

Центральна частина дашборду містить інтерактивний лінійний графік «Динаміка доходів», побудований за допомогою бібліотеки Chart.js, який візуалізує фінансові тренди підприємства (Рис. 3.10). Під блоком аналітики розміщено таблицю «Останні замовлення», де адміністратор має змогу миттєво змінювати статуси виконання робіт (наприклад, перевести з «В роботі» на «Готово до видачі»). Зміна статусів відбувається асинхронно за допомогою технології AJAX (Fetch API), що дозволяє оновлювати базу даних без перезавантаження сторінки.

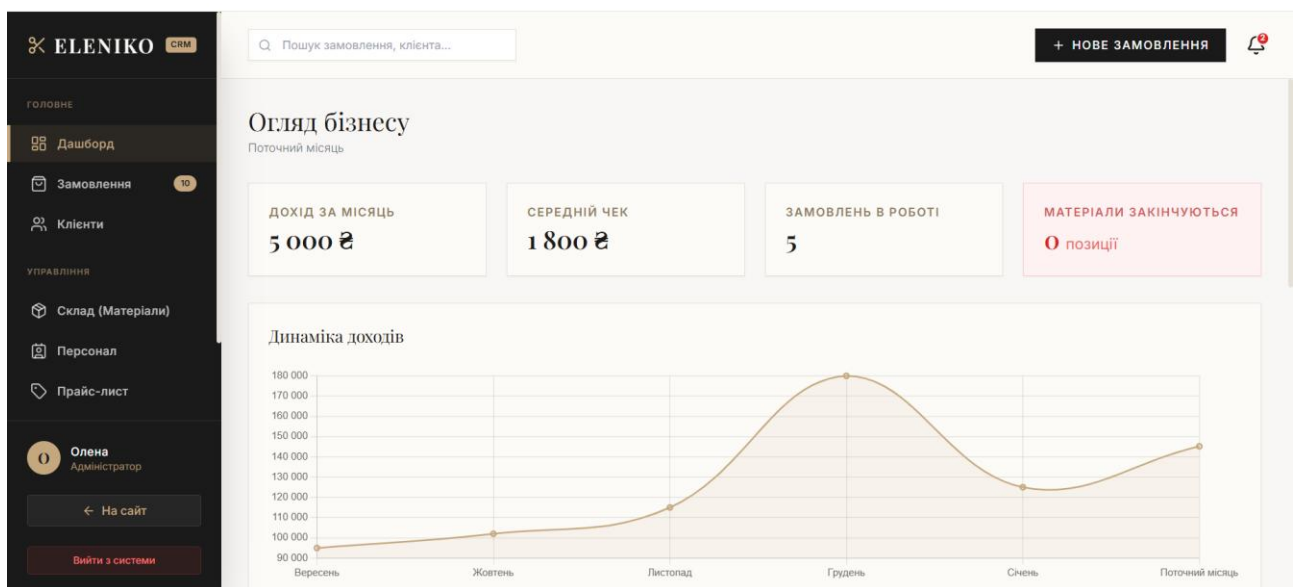


Рис.3.10 Панель адміністратора з динамічними KPI та графіком

Окремим функціональним елементом управління є механізм оформлення нового замовлення. Для забезпечення безперервного користувацького досвіду (UX), цей процес реалізовано через інтерактивне модальне вікно (Pop-up). Воно дозволяє менеджеру швидко обрати клієнта та призначити майстра з динамічних списків, підтягнутих із суміжних таблиць бази даних, встановити дедлайн і зафіксувати попередню вартість послуги (Рис. 3.11).

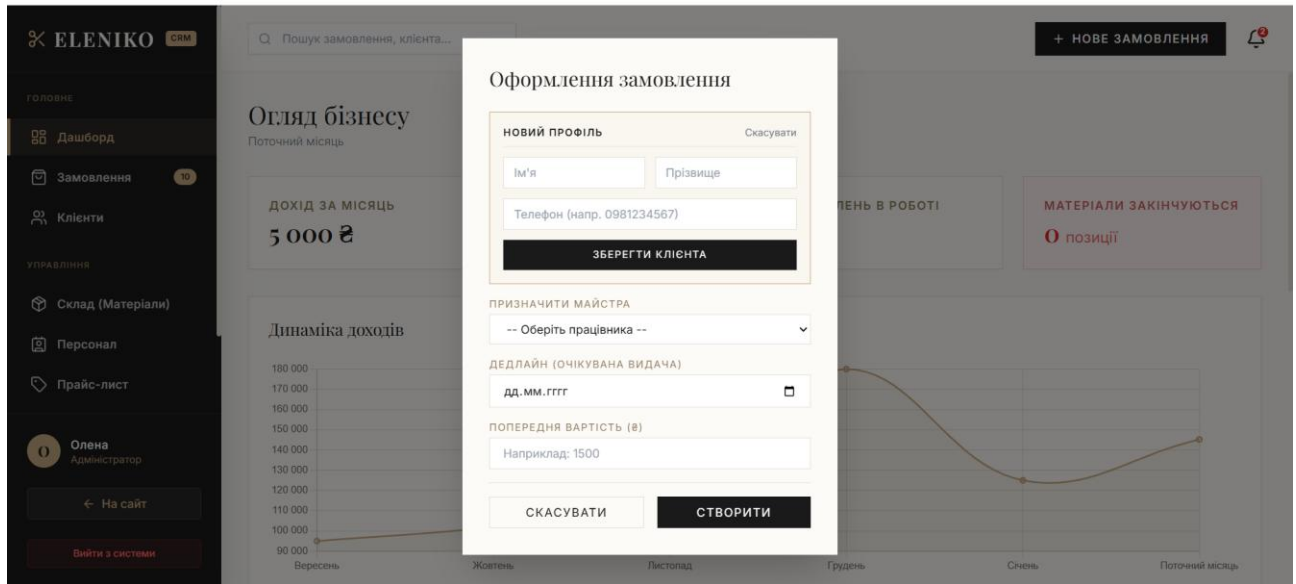


Рис.3.11 Інтерактивне модальне вікно створення нового замовлення

Крім того, для оптимізації роботи персоналу впроваджено патерн «безшовного створення профілю» (In-line creation). Якщо до майстерні звертається новий замовник, адміністратору не потрібно переривати процес оформлення та переходити до окремого модуля управління клієнтами. Завдяки розробленій архітектурі, менеджер може ввести базові контактні дані (ім'я та телефон) безпосередньо у модальному вікні. Система за допомогою асинхронного POST-запиту миттєво генерує запис у таблиці Customer і автоматично прив'язує новоствореного клієнта до поточного замовлення. Такий підхід не лише економить час, а й гарантує цілісність бази даних без ускладнення інтерфейсу.

3.2. Програмна реалізація клієнтської частини (Frontend): стилізація та інтерактивність

Візуальна частина веб-додатка побудована на утилітарному CSS-фреймворку Tailwind CSS та бібліотеці іконок Lucide. Підхід utility-first дозволяє не писати окремих CSS-файлів: усі стилі оголошуються безпосередньо в атрибутах class HTML-елементів. Кастомна кольорова палітра бренду (brand-

dark, brand-gold, brand-light) визначена через `tailwind.config` і використовується послідовно у всіх шаблонах, забезпечуючи єдиний преміальний стиль. Адаптивність інтерфейсу реалізована за допомогою стандартних брейкпоінтів (responsive design utilities) фреймворку Tailwind CSS [14], що дозволяє обійтися без написання додаткових медіа-запитів.

За динаміку інтерфейсу на стороні клієнта відповідає JavaScript без зовнішніх фреймворків. Він забезпечує фільтрацію карток портфоліо за атрибутами `data-category`, відкриття/закриття мобільного меню та ініціалізацію іконок через `lucide.createIcons()`. Уся клієнтська логіка вбудована безпосередньо в HTML-шаблони Jinja2, що відповідає архітектурі MPA (Multi-Page Application) на Flask: кожна сторінка - окремий маршрут, а JavaScript лише доповнює її поведінку.

Найбільш комплексним елементом клієнтської логіки є скрипт онлайн-калькулятора та лідогенерації (Лістинг 3.1). Реалізовано принцип розподілу відповідальності: базова вартість послуг передається сервером з таблиці `Services` через Jinja2-шаблон, тоді як розрахунки (коефіцієнти типу тканини `fabricMultiplier`, знижка за тип послуги, націнка `urgentCheck +50%`) виконуються у браузері без зайвих запитів до сервера. Додатково реалізовано лідогенерацію: кнопка «Записатись» автоматично формує персоналізоване повідомлення і підставляє його у форму зворотного зв'язку.

Лістинг 3.1 Алгоритм роботи онлайн-калькулятора

```
function calculatePrice() {
    const itemSelect = document.getElementById('service_type');
    let basePrice = parseFloat(itemSelect.value) || 0;

    const serviceType =
document.querySelector('input[name="service"]:checked').value;

    if (serviceType === 'repair') {
        basePrice = basePrice * 0.3;
    } else if (serviceType === 'upcycling') {
        basePrice = basePrice * 1.2;
    }

    const fabricMultiplier =
parseFloat(document.getElementById('fabricSelect').value) || 1;
```

```

    let total = basePrice * fabricMultiplier;

    const isUrgent = document.getElementById('urgentCheck').checked;
    if (isUrgent) {
        total = total * 1.5; // Націнка за терміновість
    }

    document.getElementById('totalPrice').innerText =
Math.round(total).toLocaleString('uk-UA');
    }

    (Scroll + Auto-fill)
    document.getElementById('btnToOrder').addEventListener('click',
function() {
    const serviceSelect = document.getElementById('service_type');
    const serviceName =
serviceSelect.options[serviceSelect.selectedIndex].text;
    const price = document.getElementById('totalPrice').innerText;

    let autoMessage = '';
    if (serviceSelect.value !== "0") {
        autoMessage = `Вітаю! Мене цікавить послуга: ${serviceName}.
Орієнтовна вартість з калькулятора: ${price} ₪. Хочу записатися на
консультацію.`;
    } else {
        autoMessage = `Вітаю! Хочу записатися на консультацію щодо
пошиття одягу.`;
    }

    const messageBox = document.getElementById('contactMessage');
    if (messageBox) {
        messageBox.value = autoMessage;
        messageBox.scrollIntoView({ behavior: 'smooth', block: 'center'
});

        messageBox.classList.add('ring-4', 'ring-brand-gold/30');
        setTimeout(() => messageBox.classList.remove('ring-4', 'ring-
brand-gold/30'), 2000);
    }
});
calculatePrice();

```

3.3. Програмна реалізація серверної частини (Backend)

Серверна архітектура відповідає за всю приховану бізнес-логіку ательє, забезпечуючи безпеку даних та їх коректне збереження.

Серверна частина системи написана мовою програмування Python з використанням легкого та потужного мікрофреймворку Flask. Цей вибір обґрунтований високою швидкістю розробки, архітектурною гнучкістю та наявністю розгалужених інструментів для інтеграції з реляційними базами даних.

Маршрутизація (Routing): згідно з архітектурними стандартами фреймворку [13], Flask забезпечує зручну обробку HTTP-запитів за допомогою декораторів `@app.route()`. Сервер приймає запити від клієнта (наприклад, перехід на сторінку "Послуги", вхід у систему або відправка даних з форми авторизації), визначає відповідну Python-функцію-обробник і повертає кінцевий згенерований HTML-шаблон за допомогою вбудованої функції `render_template`.

Інтеграція з базою даних: для встановлення з'єднання та взаємодії з СУБД MySQL використовується офіційна бібліотека `mysql-connector-python`. В межах маршруту сервер ініціює з'єднання з базою даних, виконує необхідні SQL-запити (наприклад, читання збережених мірок, створення нового запису замовлення або оновлення статусу виконання) та безпечно закриває сесію. Використання параметризованих запитів гарантує надійний захист системи від SQL-ін'єкцій.

Система авторизації та управління доступом: для безпечного розподілу користувачів у системі впроваджено архітектуру управління доступом на основі ролей (Role-Based Access Control - RBAC). Замість створення окремих сторінок входу для персоналу та клієнтів, реалізовано єдину точку входу (Single Sign-On concept).

Серверна логіка маршруту `/login` реалізує двогілкову авторизацію методом POST. Адміністратор ідентифікується за фіксованими обліковими даними (MVP-підхід). Клієнт автентифікується динамічно: система виконує параметризований SELECT-запит до таблиці `Customer` за полем `email`, і за збігу записує до Flask-сесії реальний `customer_id` знайденого запису. Це забезпечує персоналізоване відображення замовлень у кабінеті. Залежно від ролі (`admin` або `client`) маршрут перенаправляє до `/admin` або `/cabinet` відповідно. Наступний лістинг демонструє реалізацію концепції RBAC на бекенді:

Лістинг 3.2. Логіка автентифікації та управління сесіями у Flask


```

@app.route('/login', methods=['GET', 'POST'])
def login():
    # Якщо користувач вже залогінений, перенаправляємо його
    if 'user_role' in session:
        if session['user_role'] == 'admin':
            return redirect(url_for('admin_crm'))
        else:
            return redirect(url_for('cabinet'))

    error = None
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        if username == 'admin' and password == '123':
            session['user_role'] = 'admin'
            return redirect(url_for('admin_crm'))

        else:
            try:
                conn = mysql.connector.connect(**db_config)
                cur = conn.cursor(dictionary=True)
                cur.execute("SELECT customer_id, first_name FROM Customer WHERE
email = %s", (username,))
                user = cur.fetchone()

                cur.close()
                conn.close()

                if user:
                    session['user_role'] = 'client'
                    session['customer_id'] = user['customer_id']
                    session['customer_name'] = user['first_name']
                    return redirect(url_for('cabinet'))
                else:
                    error = "Користувача з таким email не знайдено!"

            except Exception as e:
                print("Помилка логіну:", e)
                error = "Помилка підключення до бази даних."

    return render_template('login.html', error=error)
@app.route('/logout')
def logout():
    session.clear() # Повне очищення сесії з міркувань безпеки
    return redirect(url_for('index'))

```

Особливістю адміністративної панелі є розвинена система КРІ-візуалізації. Серверний маршрут `/admin` виконує чотири агрегуючих SQL-запити: `SUM(amount)` з `Payments` (загальний дохід), `AVG(total_amount)` з `Orders` (середній чек), `COUNT` активних замовлень та `COUNT` матеріалів з залишком менше 5 одиниць. Отримані значення передаються у шаблон `crm.html` через словник `stats` і відображаються на картках дашборду у форматі з пробілами-розрядниками.

Центральним елементом дашборду є лінійний графік динаміки доходів, реалізований бібліотекою Chart.js у шаблоні crm.html. Сервер формує два масиви - chart_labels (місяці) і chart_values (суми) - SQL-запитом з агрегацією за 6 місяців і передає їх у шаблон. Jinja2-фільтр tojson підставляє їх безпосередньо у JavaScript-масиви. Тип графіка- line з tension: 0.4 для плавного відображення тренду та заповненням площі під кривою (fill: true)

Лістинг 3.3. Логіка ініціалізації аналітичного графіка доходів у CRM

```
const ctxRev = document.getElementById('revenueChart');
if (ctxRev) {
    // Безпечна конвертація шаблонної змінної Jinja2 у числовий формат
    let dynamicRevenue = parseInt("{} stats.revenue {}".replace(/\s/g, '')) ||
0;

    new Chart(ctxRev.getContext('2d'), {
        type: 'line',
        data: {
            labels: ['Вересень', 'Жовтень', 'Листопад', 'Грудень', 'Січень',
'Поточний місяць'],
            datasets: [{
                label: 'Дохід (€)',
                data: [95000, 102000, 115000, 180000, 125000, dynamicRevenue],
                borderColor: '#C5A880',
                backgroundColor: 'rgba(197, 168, 128, 0.1)',
                borderWidth: 2,
                fill: true,
                tension: 0.4
            }]
        },
        options: {
            responsive: true,
            maintainAspectRatio: false,
            plugins: { legend: { display: false } }
        }
    });
}
```

3.4. Реалізація модуля аналітичних звітів

Для повного задоволення технічного завдання (підрозділ 1.1), зокрема вимоги щодо “формування аналітичних звітів” (прецедент адміністратора, розділ 1.3), реалізовано окремий маршрут /admin/reports та шаблон reports.html. Ця сторінка підключена до бокової панелі CRM і відображає результати виконання

п'яти аналітичних SQL-запитів з розділу 2.5 у вигляді інтерактивних вкладок з таблицями та графіком (Рис.3.12)

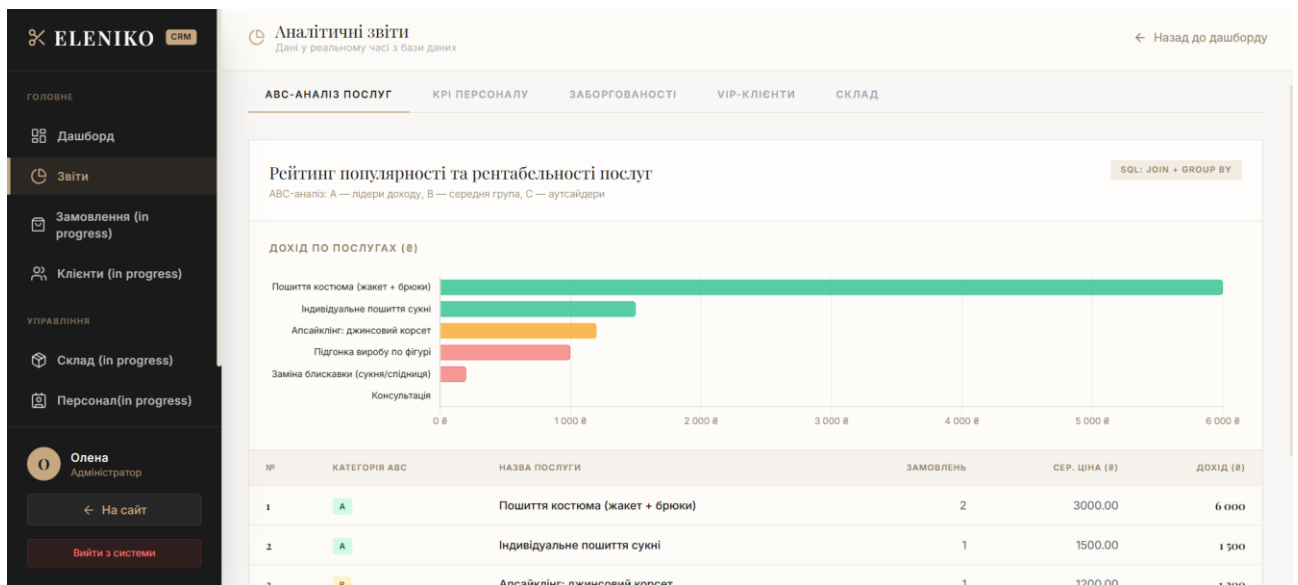


Рис. 3.12 Модуль аналітичних звітів

Сторінка звітів складається з п'яти вкладок: ABC-аналіз послуг (горизонтальна Bar-діаграма Chart.js + таблиця з автоматичними бейджами A/B/C), KPI персоналу (таблиця з прогрес-баром ефективності кожного майстра на основі LEFT JOIN), Заборгованості клієнтів (агрегація через COALESCE + HAVING з підсумковою карткою загального боргу), VIP-клієнти (рейтинг на основі CTE-запиту) та Управління складом (кольорове кодування статусу: “Критично” / “Мало” / “В нормі”). Усі дані передаються через шаблонний рушій Jinja2 безпосередньо з бази даних.

Лістинг 3.4. Серверний маршрут /admin/reports у Flask

```
@app.route('/admin/reports')
def admin_reports():
    if session.get('user_role') != 'admin':
        return redirect(url_for('login')) # Захист RBAC
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor(dictionary=True)
    # Запит 1: ABC-аналіз послуг (JOIN + GROUP BY + ORDER BY total_revenue)
    cursor.execute("""SELECT s.service_name, SUM(oi.quantity) AS total_orders,
        SUM(oi.quantity * s.base_price) AS total_revenue FROM Order_Items oi
        JOIN Services s ON oi.service_id = s.service_id
        WHERE o.status = 'Delivered' GROUP BY s.service_id ORDER BY
        total_revenue DESC""")
    services_report = cursor.fetchall()
```

```
return render_template('reports.html', services_report=services_report, ...)
```

Таким чином, аналітичні SQL-запити, спроектовані у розділі 2.5, тепер повністю інтегровані у веб-інтерфейс системи: адміністратор може переглядати результати кожного з них без необхідності звертатися до СУБД Workbench.

3.5. Хостинг та розгортання проєкту

Завершальним етапом створення системи є її виведення у виробниче середовище. Код проєкту контролюється за допомогою системи Git та зберігається у віддаленому репозиторії на GitHub.

Оскільки інформаційна система ательє має повноцінний динамічний бекенд на Python/Flask та підключену базу даних, для розгортання використовуються професійні хмарні платформи (наприклад, Railway, Heroku або PythonAnywhere). Сервер додатку та СУБД MySQL працюють у зв'язці, а всі конфіденційні змінні (паролі від бази даних, секретні ключі для сесій Flask) зберігаються виключно у прихованих змінних оточення (Environment Variables).

Обов'язковою вимогою є конфігурація SSL-сертифіката для підтримки протоколу HTTPS. Це гарантує повне шифрування трафіку між клієнтом та сервером, що є критично необхідним при передачі персональних даних (анатомічних мірок) замовників та обробці авторизаційних даних.

ВИСНОВКИ

У результаті виконання курсової роботи досягнуто поставленої мети - здійснено комплексне дослідження, проєктування та практичну реалізацію інформаційної системи для автоматизації бізнес-процесів кравецької майстерні, орієнтованої на сучасні вимоги ринку та цифрову поведінку споживачів.

Проведений аналіз ринку швейних послуг та результати емпіричного дослідження дозволили обґрунтувати високий попит на послуги ремонту, індивідуального пошиття та апсайклінгу, а також виявити ключові очікування клієнтів щодо цифровізації сервісу. Доведено, що критичними факторами конкурентоспроможності є прозорість ціноутворення, доступність онлайн-взаємодії (запис, оплата, трекінг замовлень) та персоналізований підхід до обслуговування.

У межах роботи було детально змодельовано бізнес-процеси майстерні із застосуванням BPMN-нотації, що дозволило чітко структурувати взаємодію між основними учасниками процесу (клієнтом, менеджером і майстром), виявити вузькі місця та закласти основу для їх подальшої оптимізації. Побудована UML-модель прецедентів забезпечила формалізацію функціональних вимог та логічний розподіл ролей користувачів, а DFD-діаграма - узгодження потоків даних із внутрішньою архітектурою системи.

На основі сформованих вимог спроектовано нормалізовану реляційну базу даних, що відповідає третій нормальній формі та забезпечує цілісність, несуперечливість і масштабованість даних. Запропонована структура ефективно підтримує специфічні потреби ательє, зокрема збереження антропометричних мірок, облік матеріалів, управління замовленнями та автоматизований розрахунок фінансових показників. Реалізовані SQL-запити доводять можливість отримання аналітичної інформації для прийняття управлінських рішень у режимі реального часу.

Практична частина роботи підтвердила життєздатність розробленої концепції: створено інформаційну систему із використанням сучасного

технологічного стеку (Python/Flask, MySQL, HTML, Tailwind CSS, JavaScript), який інтегрує клієнтський інтерфейс та серверну бізнес-логіку в єдину систему. Реалізовано ключові функціональні модулі: публічний веб-портал з інтерактивним калькулятором, особистий кабінет клієнта з можливістю трекінгу замовлень та CRM-панель адміністратора з аналітичними інструментами.

Упровадження запропонованої інформаційної системи дозволяє суттєво знизити вплив людського фактора, автоматизувати рутинні операції, підвищити ефективність використання ресурсів та покращити якість клієнтського сервісу. Таким чином, розроблений програмний продукт формує надійну цифрову інфраструктуру для масштабування бізнесу та підвищення його конкурентоспроможності.

Перспективи подальшого розвитку системи полягають у розширенні функціоналу шляхом інтеграції платіжних сервісів, впровадження мобільного застосунку, використання інструментів бізнес-аналітики (BI) та застосування елементів штучного інтелекту для прогнозування попиту і персоналізації пропозицій.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Valuing Our Clothes: the cost of UK fashion / Waste and Resources Action Programme (WRAP). 2017. URL: <https://www.wrap.ngo/resources/report/valuing-our-clothes-cost-uk-fashion>
2. Савчук А. О. Звіт про навчальну практику на тему: «Дослідження вимог користувачів для сайту майстерні з пошиття та ремонту одягу». Львів : ЛНУ ім. Івана Франка, 2026. 24с.
3. What Are the Stages of Business Process Management? URL: <https://www.highgear.com/blog/what-are-the-stages-of-business-process-management/>
4. Як будувати UML-діаграми. URL: <https://dou.ua/forums/topic/40575/>
5. Діаграми потоків даних (Data Flow Diagrams). URL: <https://www.maxzosim.com/data-flow-diagrams/>
6. Visual Studio Code. URL: <https://code.visualstudio.com/>
7. Що таке HTML5? URL: https://w3schoolsua.github.io/html/html5_intro.html#gsc.tab=0
8. A utility-first CSS framework for rapidly building custom designs. URL: <https://v1.tailwindcss.com/>
9. JavaScript Підручник. Основи веб програмування. URL: <https://w3schoolsua.github.io/js/index.html#gsc.tab=0>
10. Python Підручник. URL: <https://w3schoolsua.github.io/python/index.html#gsc.tab=0>
11. Опис основ нормалізації бази даних / Microsoft Learn. URL: <https://learn.microsoft.com/uk-ua/office/troubleshoot/access/database-normalization-description>
12. MySQL 8.0 Reference Manual. Chapter 11 Data Types / Oracle Corporation. URL: <https://dev.mysql.com/doc/refman/8.0/en/data-types.html>
13. Flask Documentation / Pallets Projects. URL: <https://flask.palletsprojects.com/>

14. Responsive Design / Tailwind CSS. URL: <https://tailwindcss.com/docs/responsive-design>
15. Business Process Model and Notation (BPMN). Version 2.0.2 / Object Management Group. URL: <https://www.omg.org/spec/BPMN/2.0.2/PDF>